

VidFetch: seguimiento de robots en video

Uso, funcionamiento y código de la aplicación

Dylan Frigerio
ITBA

8 de octubre de 2026

Resumen

VidFetch mide la posición, la velocidad, la orientación y la velocidad angular de N robots circulares filmados desde arriba. Detecta cada robot con un filtro adaptado a su forma (disco claro rodeado de un anillo oscuro) y `trackpy.locate`, lo sigue con N fijo mediante asignación óptima y predicción, deriva la velocidad con un filtro de Savitzky–Golay y obtiene el giro comparando firmas polares en el dominio de Fourier. Este informe explica para qué sirve, cómo está organizado, qué bibliotecas usa y qué hace cada parte del código. Los resultados de los ensayos están en un informe aparte.

Índice

1. Qué es y para qué sirve	3
1.1. Requisitos y ejecución	3
1.2. Organización del código	3
1.3. La interfaz	4
1.4. Archivos que lee y escribe	5
2. Bibliotecas utilizadas	5
3. Escala de tiempo: videos acelerados	7
4. Metodología de medición	8
4.1. Flujo de procesamiento	8
4.2. Sistema de referencia y unidades	9
4.3. Calibración automática del tamaño	10
4.4. Detección	10
4.5. Seguimiento con N fijo	12
4.6. Velocidad	13
4.7. Orientación y velocidad angular	14
4.8. Recorte temporal	16
4.9. Incertidumbres y limitaciones	16
5. Procedimiento y exportaciones	17
5.1. Analizar un video	17
5.2. Generar el informe de ensayos	17
5.3. Archivos exportados	18

1. Qué es y para qué sirve

VidFetch es una aplicación de escritorio en Python que mide, a partir de un video filmado desde arriba, el movimiento de N robots circulares dentro de una arena. Para cada robot y cada cuadro entrega la posición del centro, el vector velocidad, la orientación acumulada θ y la velocidad angular ω , en unidades SI y en tiempo real aunque el video esté acelerado.

Usos previstos:

- **Preparar el video:** recortar el tramo útil y el rectángulo de la arena, y guardar el recorte (archivos VC_).
- **Probar la detección** sobre cuadros sueltos y ajustar parámetros a mano o con la calibración automática.
- **Seguir** N robots fijos durante todo el ensayo, revisar los cuadros dudosos y corregir centros a mano.
- **Exportar** los datos (CSV), un resumen por robot, el video anotado y la sesión de análisis para retomarla sin volver a procesar.
- **Generar los informes** de ensayos a partir de los CSV (sección 5).

1.1. Requisitos y ejecución

Requiere Python 3.10 o superior y las bibliotecas de `requirements.txt` (sección 2). Desde la carpeta `software/vidfetch` del repositorio:

```
pip install -r requirements.txt
python main.py
```

Las tareas largas (análisis, exportaciones) corren en un hilo aparte, con barra de progreso y botón de cancelar, de modo que la ventana no se congela.

1.2. Organización del código

El código separa la lógica de cálculo (`core/`, sin dependencias de la interfaz) de la interfaz gráfica (`gui/`). Así, todo el cálculo puede usarse desde un script, y la interfaz solo llama a funciones de `core/` (tabla 1).

Tabla 1: Archivos del programa y su función (rutas relativas a `software/vidfetch`).

Archivo	Líneas	Función
<code>main.py</code>	18	punto de entrada: crea la aplicación Qt y la ventana principal
<code>core/models.py</code>	47	Roi (rectángulo) y TrimRange (tramo de cuadros)
<code>core/video_io.py</code>	169	lectura de video con acceso aleatorio, exportación de video recortado o anotado
<code>core/processing.py</code>	94	operaciones de imagen encadenables (grises, brillo/contraste, desenfoque, umbral, Canny)
<code>core/detection.py</code>	182	detección de robots: filtro de anillo, <code>trackpy.locate</code> , cobertura del anillo
<code>core/calibration.py</code>	230	estimación automática del radio del robot y de todos los umbrales
<code>core/analysis.py</code>	116	recorrido del video cuadro a cuadro y tabla de candidatos
<code>core/tracking.py</code>	708	seguimiento con N fijo, anclas manuales, huecos, sesión <code>.npz</code> , dibujo
<code>core/rotation.py</code>	139	firmas polares y orientación acumulada θ
<code>core/kinematics.py</code>	250	velocidad, ω , escala SI y tablas de exportación
<code>gui/main_window.py</code>	522	ventana, menús, diálogos de archivo y tareas en segundo plano
<code>gui/state.py</code>	71	estado compartido: video, tramo, rectángulo, escala de tiempo
<code>gui/tabs.py</code>	501	pestañas <i>Original</i> , <i>Recortar</i> y <i>Procesar</i>
<code>gui/tracking_tab.py</code>	498	pestaña <i>Seguimiento</i>
<code>gui/player.py, frame_view.py</code>	362	reproductor y visor con zoom y edición del rectángulo
<code>gui/export_worker.py</code>	35	hilo de trabajo (QThread) para tareas largas
<i>En <code>software/analisis_video/</code>:</i>		
<code>generar_informe.py</code>	548	sección de resultados de un ensayo a partir de su CSV
<code>comparar_ensayos.py</code>	530	comparación entre ensayos y contraste con el sensor de presión

1.3. La interfaz

La ventana tiene cuatro pestañas que comparten el mismo video, tramo y rectángulo, guardados en un único objeto de estado (`gui/state.py`): lo que se define en una se refleja en las demás.

1. **Original:** reproduce el video y fija la escala de tiempo (sección 3). La etiqueta del reproductor muestra el cuadro y el tiempo real.
2. **Recortar:** tramo temporal (cuadro de inicio y fin) y rectángulo de análisis, que se dibuja o se mueve sobre la imagen; permite forzar un cuadrado de lado fijo y guardar el recorte.
3. **Procesar:** aplica operaciones de imagen y prueba la detección cuadro a cuadro (parámetros, botón *Calibrar tamaño automáticamente*). Exporta posiciones sueltas o vinculadas con `trackpy.link` y el video procesado.
4. **Seguimiento:** análisis completo con N fijo, lista de cuadros con problemas, corrección manual del centro, parámetros de cinemática (ventana de suavizado, diámetro real, corrección de video espejado), superposición de velocidad y orientación, y exportaciones.

1.4. Archivos que lee y escribe

Tabla 2: Formatos de archivo.

Extensión	Sentido	Contenido
<code>.mp4</code> <code>.avi</code> <code>.mov</code> <code>.mkv</code> <code>.m4v</code> <code>.wmv</code>	entrada	video a analizar (cualquier formato que lea OpenCV/FFmpeg)
<code>.mp4</code> (<code>mp4v</code>), <code>.avi</code> <code>.csv</code>	salida	video recortado, procesado o con el seguimiento dibujado
	salida	posiciones de la pestaña <i>Procesar</i> ; seguimiento completo en formato largo o ancho; resumen por robot (columnas en la tabla 7)
<code>.npz</code>	entrada y salida	sesión de análisis: candidatos, firmas de rotación, anclas manuales y parámetros (NumPy comprimido, sin <code>pickle</code>)
<code>.tex</code> , <code>.csv</code> , <code>.png</code>	salida	secciones de informe generadas por los scripts de <code>software/analisis_video/</code>

La sesión `.npz` se guarda con `numpy.savez_compressed` y se carga con `allow_pickle=False`: contiene solo arreglos numéricos y un texto JSON con los parámetros, por lo que abrir un archivo ajeno no ejecuta código.

2. Bibliotecas utilizadas

Tabla 3 resume qué se usa de cada biblioteca. La regla general es usar funciones compiladas (OpenCV, SciPy, NumPy vectorizado) para todo lo que se repite por píxel o por cuadro, y dejar en Python solo la lógica de decisión.

Tabla 3: Bibliotecas, funciones usadas y para qué. Versiones probadas entre paréntesis.

Biblioteca	Funciones	Uso
OpenCV (cv2 5.0)	VideoCapture, CAP_PROP_* VideoWriter, VideoWriter_fourcc cvtColor, resize (INTER_AREA) filter2D warpPolar GaussianBlur, threshold, Canny, convertScaleAbs, dilate circle, line, arrowedLine, putText, ellipse	abrir el video, leer metadatos (cuadros, fps, tamaño) y saltar a un cuadro escribir videos recortados o anotados pasar a grises y reducir la imagen a la escala de análisis convolución con los núcleos de disco y anillo (filtro de anillo, sección 4.4) imagen en coordenadas polares alrededor de cada centro: cobertura del anillo y firma de rotación operaciones de la pestaña <i>Procesar</i> y calibración dibujo de la superposición y del video anotado
trackpy (0.7)	locate link, filter_stubs	máximos locales del mapa de respuesta con centroide subpíxel (sección 4.4) vinculación simple de trayectorias en la pestaña <i>Procesar</i> (no se usa en el seguimiento con N fijo)
SciPy (1.18)	optimize.linear_sum_assignment spatial.cKDTree signal.savgol_filter sparse.coo_matrix, sparse.linalg.spsolve	asignación óptima candidato-robot (algoritmo húngaro) búsqueda de vecinos para confirmar robots nuevos por persistencia derivadas suavizadas: velocidad y ω mínimos cuadrados del ángulo acumulado
NumPy (2.5)	fft.fft, fft.ifft arreglos, interp, savez_compressed, load	armónicos de la firma de rotación y correlación circular todo el cálculo numérico, interpolación de huecos y sesiones .npz
pandas (3.0)	DataFrame, pivot, to_csv, read_csv	tablas de candidatos y de resultados, formato ancho, CSV
PySide6 (Qt 6.11)	QMainWindow, QTabWidget, widgets de formulario QThread + señales QTimer QImage, QPainter	interfaz tareas largas sin congelar la ventana reproducción del video visor con zoom y edición del rectángulo
Matplotlib	imshow, histogram2d (NumPy)	solo en comparar_ensayos.py : mapas de ocupación

Por qué trackpy solo para localizar. `trackpy.locate` es robusto para encontrar máximos y refinarlos a precisión subpíxel, y por eso se usa. Su vinculador (`link`) no sabe que la cantidad de robots es fija: ante un robot que desaparece unos cuadros crea una trayectoria nueva, y ante un falso positivo, también. Por eso el seguimiento del ensayo usa un algoritmo propio con N fijo (sección 4.5), y `link` queda solo como opción rápida en *Procesar*.

Por qué OpenCV para el video. `cv2.VideoCapture` usa FFmpeg internamente: lee cualquier formato habitual y permite acceso aleatorio por número de cuadro, que necesitan el reproductor, la calibración (cuadros saltados) y la revisión de cuadros con problemas.

3. Escala de tiempo: videos acelerados

Dos frecuencias distintas. Hay que distinguir

- f_a : frecuencia de *reproducción* del archivo (29,97 cuadros/s). Solo define a qué velocidad se ve el video; no tiene significado físico.
- f_r : frecuencia de *captura* en tiempo real, es decir, cuántos cuadros corresponden a 1 s del ensayo. Es la única que entra en los cálculos.

Con el cuadro n contado desde el inicio del archivo, el tiempo real y el factor de aceleración son

$$t_n = \frac{n}{f_r}, \quad \Delta t = \frac{1}{f_r}, \quad A = \frac{f_a}{f_r}. \quad (1)$$

En este informe $f_r = 3$ cuadros/s, valor definido en `config.tex` (`\CuadrosPorSegundo`): entre dos cuadros consecutivos pasan $\Delta t = 0,33$ s reales y el video se reproduce $A = 9,99$ veces más rápido que el ensayo. Todos los tiempos de este documento (gráficos, tablas y equivalencias entre cuadros y segundos) se calculan con ese valor.

Qué depende de f_r y qué no. El análisis completo trabaja en cuadros: detecta, sigue y mide ángulos cuadro por cuadro, y en ningún momento usa el tiempo. Recién al final se pasa a unidades físicas. Si s es la escala espacial [mm/px] y d/dn la derivada respecto del número de cuadro,

$$\vec{v} = s f_r \frac{d\vec{r}_{px}}{dn}, \quad \omega = f_r \frac{d\theta}{dn}. \quad (2)$$

Por lo tanto:

- posiciones, trayectorias, desplazamientos, ángulos θ y estados de seguimiento **no** dependen de f_r ;
- tiempos, velocidades y velocidades angulares son proporcionales a $1/f_r$ o a f_r : un error de f_r se traslada en la misma proporción a $|\vec{v}|$ y a ω ;
- cambiar f_r **no** obliga a repetir el análisis: en la aplicación, la cinemática se recalcula en segundos; en el informe, `generar_informe.py` reescala el CSV (sección 5).

La incertidumbre relativa de las velocidades combina la de la escala espacial y la de la temporal:

$$\frac{\delta v}{v} = \sqrt{\left(\frac{\delta s}{s}\right)^2 + \left(\frac{\delta f_r}{f_r}\right)^2}, \quad \frac{\delta \omega}{\omega} = \frac{\delta f_r}{f_r}. \quad (3)$$

Cómo se determina f_r . Los archivos de la cámara no guardan la marca de tiempo de cada cuadro (la pista de subtítulos DJI solo trae datos de exposición, uno por segundo), así que f_r no se puede leer del video. Hay tres formas de obtenerlo, de mejor a peor:

1. **Configuración de la cámara.** En un *time-lapse* $\times 10$, cada segundo de video (29,97 cuadros) cubre 10 s reales, es decir $f_r = f_a/10 = 2,997$ cuadros/s.
2. **Duración real medida.** $f_r = N_{\text{cuadros}}/T_{\text{real}}$, con T_{real} cronometrado (o tomado del registro del sensor de presión, que tiene su propio reloj y se graba en simultáneo).
3. **Duración nominal del nombre** (`_60`). No conviene: es la duración prevista, no la medida, y en los ensayos del proyecto difiere hasta 12 % de la real (ver el informe de resultados de los ensayos).

Consecuencias del *time-lapse*. Muestrear a f_r en lugar de a 30 Hz cambia lo que se puede medir:

- **Resolución temporal.** Un fenómeno más rápido que la frecuencia de Nyquist, $f_r/2 = 1,5$ Hz, no se puede resolver: un choque que dura menos de $\Delta t = 0,33$ s aparece, a lo sumo, como un salto entre dos cuadros.
- **Desplazamiento por cuadro.** Entre cuadros consecutivos los robots se mueven f_a/f_r veces más que en un video en tiempo real. El seguimiento acepta, como máximo, un apartamiento de $0,625 r_{\text{ext}}$ por cuadro respecto de la posición predicha, lo que equivale a un cambio de velocidad de $0,625 (D/2) f_r \approx 33$ mm/s entre dos cuadros; en los ensayos medidos la rapidez típica es de pocos mm/s, muy por debajo de ese límite.
- **Rotación por cuadro.** El giro entre dos cuadros que se comparan debe ser menor que media vuelta para no confundir su sentido: $|\omega| < 180^\circ \cdot f_r = 540^\circ/\text{s}$.
- **Parámetros en cuadros.** Las ventanas y tolerancias del método están definidas en cuadros, no en segundos, porque el detector y el seguidor trabajan cuadro a cuadro; su duración real depende de f_r (tabla 4).

Tabla 4: Parámetros del método expresados en cuadros y su equivalente en tiempo real con $f_r = 3$ cuadros/s.

Parámetro	Cuadros	Tiempo real
Paso de muestreo Δt	1	0,33 s
Ventana de Savitzky–Golay (velocidad y ω)	7	2,3 s
Hueco máximo que se interpola sin advertencia de error	15	5,0 s
Persistencia exigida a un robot nuevo (75 % de)	20	6,7 s
Comparaciones de rotación (desfasajes)	1, 2, 4, 8, 16	hasta 5,3 s
Flecha de velocidad dibujada (desplazamiento en)	7,5	2,5 s

En la aplicación. En la pestaña *Original*, el grupo *Escala de tiempo* tiene la casilla *Video acelerado (time-lapse)* y el campo $\langle f_r \rangle \text{ cuadros} = 1 \text{ s real}$ (por defecto, 3). El botón *Calcular desde la duración real...* pide la duración medida en minutos y calcula $f_r = N_{\text{cuadros}}/T$; el nombre del archivo se muestra solo como referencia, por ser nominal. Debajo se indica cuánto dura un cuadro en tiempo real, el factor A y la duración real del archivo. El reproductor sigue mostrando el video a f_a , pero la etiqueta de tiempo (*cuadro i / $N \cdot t$*) está en tiempo real. Si se cambia f_r con un seguimiento ya hecho, solo se recalcula la cinemática. Al abrir una sesión guardada (.npz) con otra escala, se aplica la escala vigente y se avisa. Desmarcar la casilla vuelve a $f_r = f_a$ (videos en tiempo real).

4. Metodología de medición

Para cada robot de la arena y en cada cuadro del video se mide la posición del centro, el vector velocidad, la orientación acumulada θ y la velocidad angular ω , con la cantidad de robots N fija. Las distancias se convierten a unidades SI con el diámetro real de los robots, $D = 3,5$ cm, y los tiempos con la frecuencia de captura f_r (sección 3). Las figuras de ejemplo de esta sección provienen de un tramo de 180 cuadros de un ensayo con 22 robots (recorte de 700×700 px), expresado con $f_r = 3$ cuadros/s (60 s reales).

4.1. Flujo de procesamiento

El análisis se divide en dos etapas (fig. 1). La etapa A es la única que lee el video; corregir un centro a mano, cambiar parámetros del seguimiento o recortar el tiempo solo vuelve a correr la etapa B, que tarda segundos.

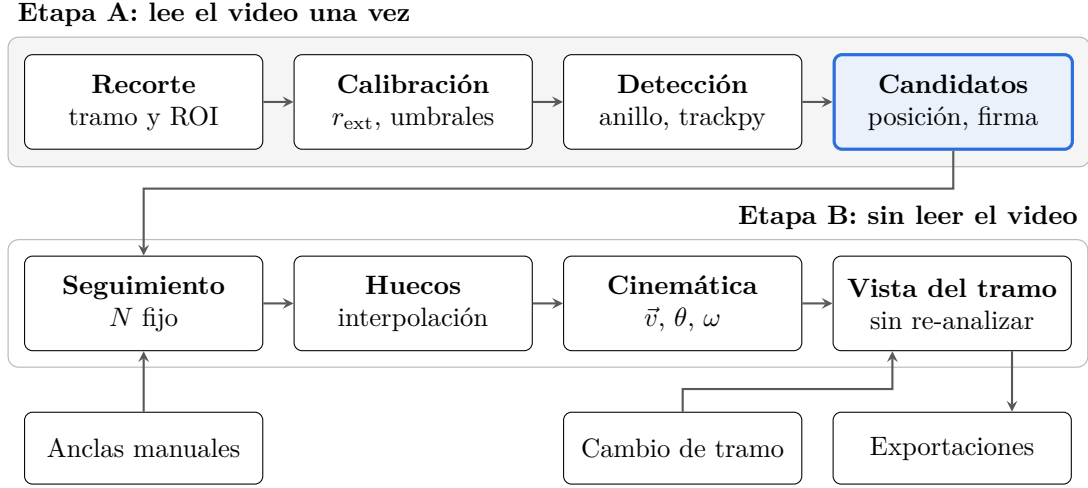


Figura 1: Flujo de procesamiento.

4.2. Sistema de referencia y unidades

Las posiciones usan los ejes de la imagen: x hacia la derecha, y hacia abajo, con origen en la esquina superior izquierda del recorte analizado. Los ángulos (dirección de la velocidad, θ y ω) son positivos en sentido antihorario visto en pantalla. La escala resulta de

$$s = \frac{D}{2r_{\text{ext}}} \quad [\text{mm/px}], \quad (4)$$

donde r_{ext} es el radio exterior del robot en píxeles medido por la calibración; su incertidumbre ($\pm 2\%$) se propaga a todas las magnitudes en metros.

Video espejado. Algunas cámaras graban la escena reflejada. Una reflexión invierte la quiralidad: un giro horario se ve antihorario y viceversa. La opción *Video espejado* de la pestaña *Seguimiento* corrige las exportaciones sin tocar el análisis, que siempre se hace sobre la imagen tal como está (así la superposición coincide con el video). Para una reflexión respecto del eje vertical de la imagen (*horizontal*), con W el ancho de la región analizada,

$$x \rightarrow W - x, \quad v_x \rightarrow -v_x, \quad \theta \rightarrow -\theta, \quad \omega \rightarrow -\omega, \quad (5)$$

y la dirección de la velocidad se recalcula; para una reflexión *vertical* se cambian y y v_y en lugar de x y v_x . El CSV lleva entonces la columna **espejo**, y las columnas **x_video_px**, **y_video_px** quedan siempre en píxeles del archivo.

Tabla 5: Magnitudes medidas.

Magnitud	Símbolo	Unidad	Definición
Tiempo real	t	s	$t = n/f_r$, con n el número de cuadro
Posición	x, y	m	centro del robot, ejes de imagen
Velocidad	v_x, v_y	m/s	$dx/dt, dy/dt$
Rapidez	$ \vec{v} $	m/s	$\sqrt{v_x^2 + v_y^2}$
Dirección	φ	°	$\text{atan2}(-v_y, v_x)$; 0° = derecha
Orientación	θ	°	giro acumulado desde el primer cuadro del tramo
Velocidad angular	ω	°/s, rad/s	$d\theta/dt$

4.3. Calibración automática del tamaño

Visto desde arriba, el robot es un disco claro (la pila) rodeado por un anillo oscuro (la placa). Se usa una plantilla con un disco de radio $r_{\text{int}} = 0,75 r_{\text{ext}}$ y un anillo hasta r_{ext} , y se barre r_{ext} : la respuesta media de los N picos más fuertes es máxima cuando la plantilla coincide con el robot real (fig. 2). A partir de r_{ext} se derivan la escala de análisis ($24/r_{\text{ext}}$, para trabajar siempre con robots de unos 24px de radio), la separación mínima ($1,5 r_{\text{ext}}$) y el radio de búsqueda del seguimiento ($0,625 r_{\text{ext}}$). Los umbrales de gris se fijan como fracción del contraste medido entre anillo y disco, por lo que no dependen de la exposición.

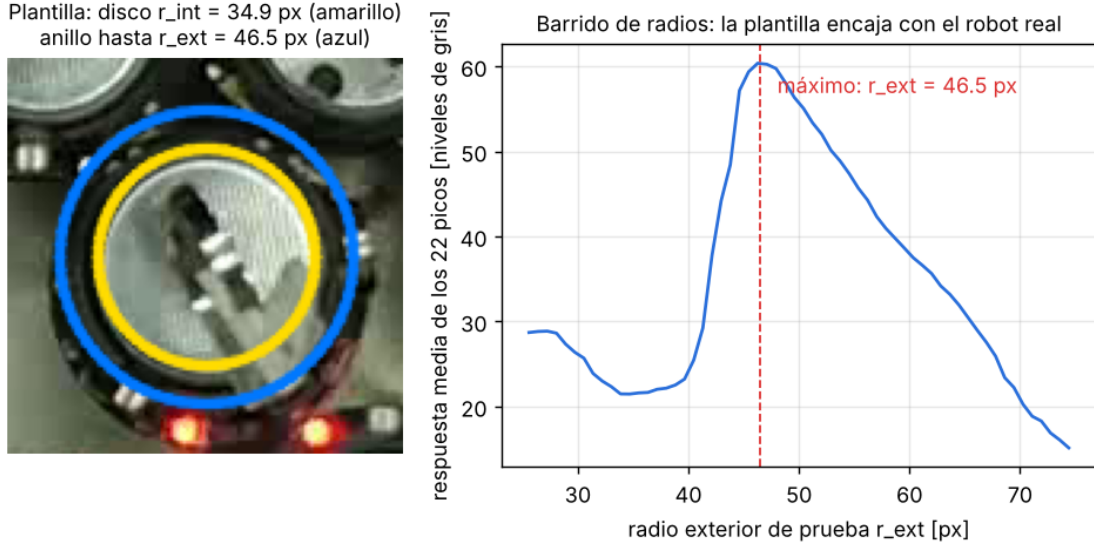


Figura 2: Calibración: plantilla sobre un robot y respuesta en función del radio de prueba.

4.4. Detección

Filtro de anillo. Para cada píxel se calculan el gris medio del disco y del anillo con núcleos circulares normalizados, y se combinan como

$$R(x, y) = (\mu_{\text{disco}} - \mu_{\text{anillo}}) \text{ clip}\left(\frac{G_{\text{máx}} - \mu_{\text{anillo}}}{G_{\text{máx}}}, 0, 1\right). \quad (6)$$

El primer factor exige contraste; el segundo, que el anillo sea oscuro en valor absoluto. R es alto solo en los centros de los robots. En el código (código 1), las medias locales son dos convoluciones `cv2.filter2D` con núcleos normalizados a suma 1, calculadas sobre la imagen ya reducida a la escala de análisis (los robots quedan con unos 24px de radio, lo que hace el costo independiente de la resolución del video).

```

1 def _disc_ring_kernels(r_in, r_out):
2     R = int(np.ceil(r_out))
3     yy, xx = np.mgrid[-R:R + 1, -R:R + 1]
4     rr = np.hypot(xx, yy)
5     disc = (rr <= r_in).astype(np.float32)
6     ring = ((rr > r_in) & (rr <= r_out)).astype(np.float32)
7     return disc / disc.sum(), ring / ring.sum() # mean filters
8
9 def robot_response(gray, p):
10     """Matched-filter map (uint8) at analysis scale."""
11     ki, kr = _disc_ring_kernels(p.r_in * p.scale, p.r_out * p.scale)
12     g = gray.astype(np.float32)
13     m_disc = cv2.filter2D(g, -1, ki, borderType=cv2.BORDER_REFLECT)
14     m_ring = cv2.filter2D(g, -1, kr, borderType=cv2.BORDER_REFLECT)
15     gate = np.clip((p.ring_gray_max - m_ring) / p.ring_gray_max, 0.0, 1.0)

```

```
16 return np.clip((m_disc - m_ring) * gate, 0, 255).astype(np.uint8)
```

Código 1: Filtro de anillo (core/detection.py).

Localización. `trackpy.locate` busca los máximos locales de R y devuelve su centroide con precisión subpíxel. Se aplica sobre R y no sobre la imagen porque la pila no es una mancha uniforme: la palanca metálica y los reflejos corren el centroide. Los argumentos salen de la calibración: el diámetro de la mancha buscada ($1,4 r_{\text{int}}$ en la escala de análisis, redondeado a impar, porque el pico de R es más angosto que el disco), la separación mínima entre centros y `minmass`. Las coordenadas devueltas se dividen por la escala para volver a píxeles del video (código 2).

```
1 f = tp.locate(img, p.trackpy_diameter(), minmass=p.minmass,
2             separation=max(1.0, p.separation * p.scale), invert=False)
3 f["x"] = f["x"] / p.scale           # back to video pixels
4 f["y"] = f["y"] / p.scale
```

Código 2: Llamada a `trackpy` (core/detection.py, función `locate`).

Validación. Un hueco de piso rodeado de robots también genera un pico, pero su “anillo” está formado por los vecinos y no es cerrado. Se trazan 360 rayos desde el centro y se exige que al menos el 82 % encuentre un píxel oscuro dentro de la banda del anillo (figs. 3 and 4). `cv2.warpPolar` desenrolla la imagen alrededor del candidato: cada fila es un rayo y cada columna un radio, de modo que la cobertura es una sola operación vectorizada (código 3).

```
1 def ring_coverage(gray, x, y, r0, r1, dark):
2     """Fraction of 360 rays whose darkest pixel in the band [r0, r1] is below `dark`."""
3     r1i, r0i = int(np.ceil(r1)), int(max(0, np.floor(r0)))
4     pol = cv2.warpPolar(gray, (r1i + 1, 360), (float(x), float(y)), r1i + 1,
5                       cv2.WARP_POLAR_LINEAR)           # rows = angle, cols = radius
6     return float((pol[:, r0i:r1i].min(axis=1) < dark).mean())
```

Código 3: Cobertura angular del anillo oscuro (core/detection.py).

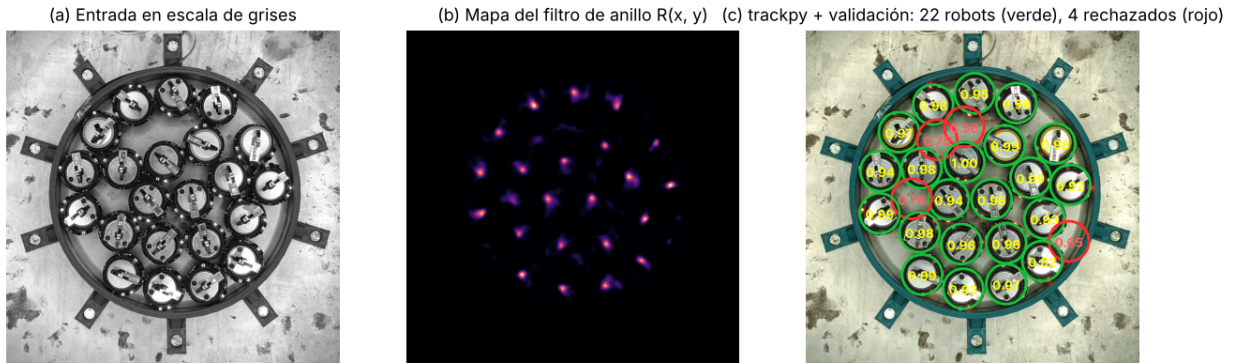


Figura 3: (a) Entrada. (b) Mapa R . (c) Candidatos con su cobertura: aceptados (verde) y huecos rechazados (rojo).

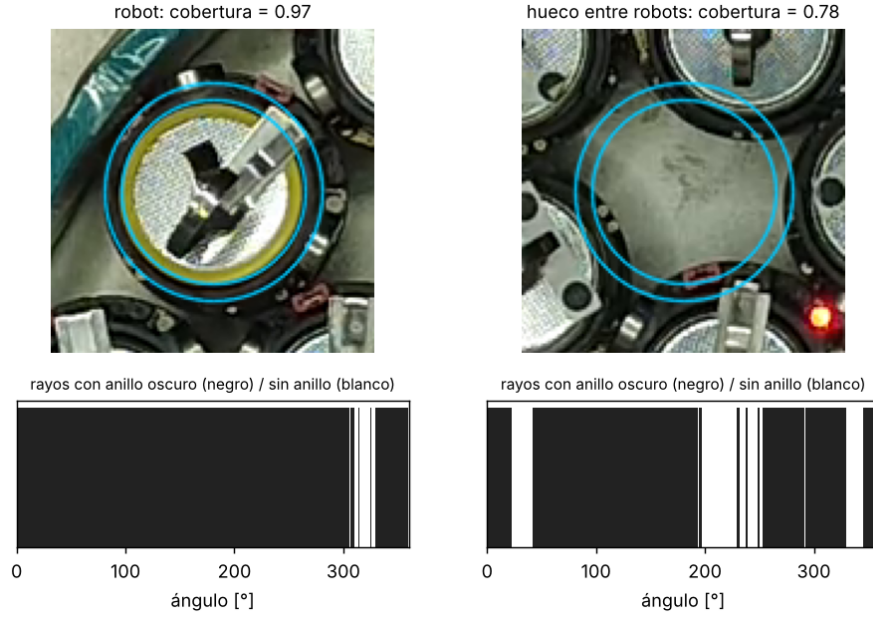


Figura 4: Cobertura del anillo: un robot cierra el anillo en casi todos los rayos; un hueco, no.

4.5. Seguimiento con N fijo

En cada cuadro se predice la posición de cada robot (última posición más velocidad media reciente) y se asignan predicciones y candidatos minimizando la distancia total (algoritmo húngaro), solo dentro del radio de búsqueda. A los robots sin pareja se les ofrecen candidatos con umbrales relajados (cobertura $\geq 0,70$ y luego $\geq 0,55$), pero solo cerca de su predicción. Un robot nuevo solo se acepta si persiste en el 75 % de los 20 cuadros (6,7s) siguientes. Los huecos acotados se interpolan linealmente y cada punto queda con un estado (tabla 6). Un centro marcado a mano (ancla) es una restricción fija desde la que el seguimiento continúa (fig. 5).

El seguimiento corre en dos etapas. La etapa A recorre el video una sola vez y guarda, para cada cuadro, todos los candidatos con un umbral permisivo (cobertura $\geq 0,45$) y su firma de rotación. La etapa B trabaja solo sobre esa tabla: parte del cuadro con mejor detección y avanza hacia adelante y hacia atrás. Cada robot guarda su última posición y una velocidad suavizada exponencialmente ($\alpha = 0,5$); la predicción extrapola esa velocidad, limitada a `max_gap` cuadros para no alejarse demasiado en un hueco. La asignación usa `scipy.optimize.linear_sum_assignment` con costo igual a la distancia, y costo prohibitivo fuera del radio de búsqueda, que crece con la cantidad de cuadros sin detección (código 4).

```

1 def predict(self, frame, max_gap):
2     dt = frame - self.f                                     # signed: also runs backwards
3     eff = np.sign(dt) * np.minimum(np.abs(dt), max_gap)    # damp long extrapolations
4     return self.xy + self.v * eff[:, None], np.abs(dt)
5
6 def _gated_assign(pred, gates, cand):
7     d = np.linalg.norm(pred[:, None, :] - cand[None, :, :], axis=2)
8     cost = np.where(d <= gates[:, None], d, _BIG)          # outside the gate = forbidden
9     r, c = linear_sum_assignment(cost)                     # Hungarian algorithm
10    return [(i, j) for i, j in zip(r, c) if cost[i, j] < _BIG]
11
12 # per frame: manual anchors first, then the strict and the relaxed coverage levels
13 for cov_thr, gate_factor, status in levels:               # (0.82, 1, DETECTED), (0.70, 0.6, RELAXED), ...
14     ks = np.flatnonzero(st.has & ~done)
15     js = np.flatnonzero(~used & (cov >= cov_thr))
16     for a, b in _gated_assign(pred[ks], base_gate[ks] * gate_factor, xy[js]):
17         ... # store position, status and candidate index; update the velocity

```

Código 4: Predicción y asignación con compuerta (core/tracking.py, resumido).

Tabla 6: Estados de cada posición (columna **status**).

Estado	Significado
detectado	asignado con el umbral estricto
detectado (umbral relajado)	asignado con cobertura 0,55–0,82 cerca de la predicción
manual	centro marcado a mano
interpolado	hueco de hasta 15 cuadros (5,0 s) entre dos detecciones (advertencia)
predicho (revisar)	hueco más largo o sin detección de un lado (error)
perdido	sin ninguna posición

Cuadro 105: robot 12 interpolado (naranja), robot 5 con huecos de prueba

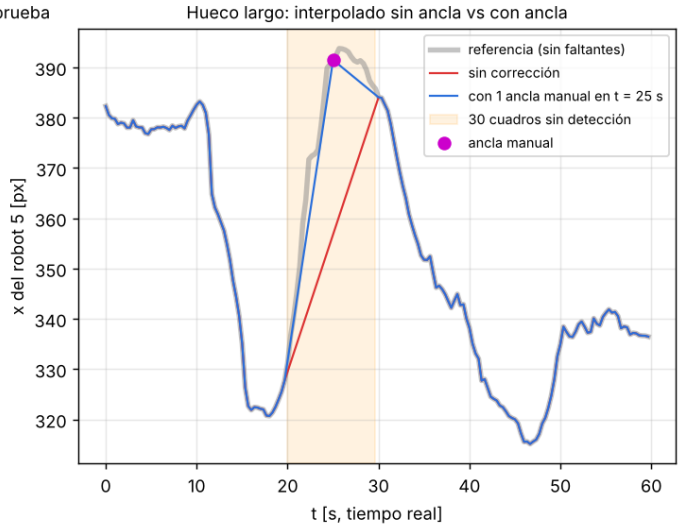
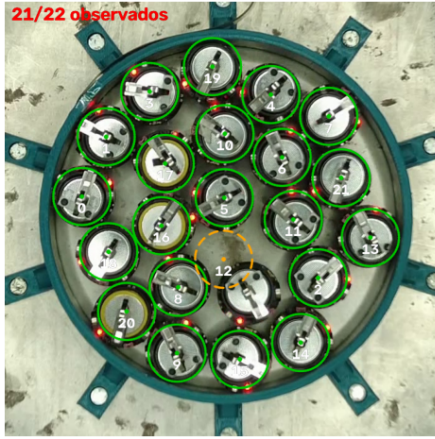


Figura 5: Faltantes simulados: robot interpolado (naranja) y efecto de una sola ancla manual en un hueco de 30 cuadros (10 s). Tiempo real con $f_r = 3$ cuadros/s.

4.6. Velocidad

El centro detectado fluctúa entre 1 y 5 px por cuadro; una diferencia finita multiplicaría ese ruido por f_r . Por eso la velocidad se obtiene con un filtro de Savitzky–Golay: para cada instante se ajusta por mínimos cuadrados un polinomio de grado 2 a los cuadros vecinos (ventana de 7 cuadros (2,3 s) por defecto) y se toma su derivada analítica,

$$v_x(t_i) = \left. \frac{d\hat{p}_i}{dt} \right|_{t_i}, \quad \hat{p}_i = \arg \min_{p \in \mathbb{P}_2} \sum_{j=-3}^3 [x(t_{i+j}) - p(t_{i+j})]^2, \quad (7)$$

con $t_{i+j} = (i+j)/f_r$, y lo mismo para v_y (fig. 6). Una ventana mayor reduce el ruido pero atenúa los cambios bruscos, como los choques. La ventana w es un número de cuadros (campo *Suavizado* de la pestaña *Seguimiento*): su duración real es w/f_r y crece si la captura es más lenta. En el código, `scipy.signal.savgol_filter` con `deriv=1` y `delta = 1/f_r` devuelve directamente la derivada por segundo real; los huecos (NaN) se rellenan por interpolación solo para filtrar y se vuelven a marcar después (código 5). La escala espacial se aplica al final: $v \text{ [m/s]} = v \text{ [px/s]} \cdot s$.

```

1 def _derivative(v, fps, p):
2     """d/dt along axis 0 for each column; NaNs are bridged by interpolation, then restored."""
3     out = np.full_like(v, np.nan, dtype=float)
4     n = len(v)
5     for k in range(v.shape[1]):
6         col = v[:, k]

```



```

7      ok = np.isfinite(col)
8      if ok.sum() < 2:
9          continue
10     idx = np.arange(n)
11     filled = np.interp(idx, idx[ok], col[ok])
12     w = _odd_window(p.window, n, p.polyorder)
13     d = savgol_filter(filled, w, p.polyorder, deriv=1, delta=1.0 / fps, mode="interp")
14     d[~ok] = np.nan
15     out[:, k] = d
16     return out

```

Código 5: Derivada temporal suavizada (core/kinematics.py).

Robot 2: trayectoria (cian) y v cada 12 cuadros (flecha = desplazamiento en 7,5 cuadros)

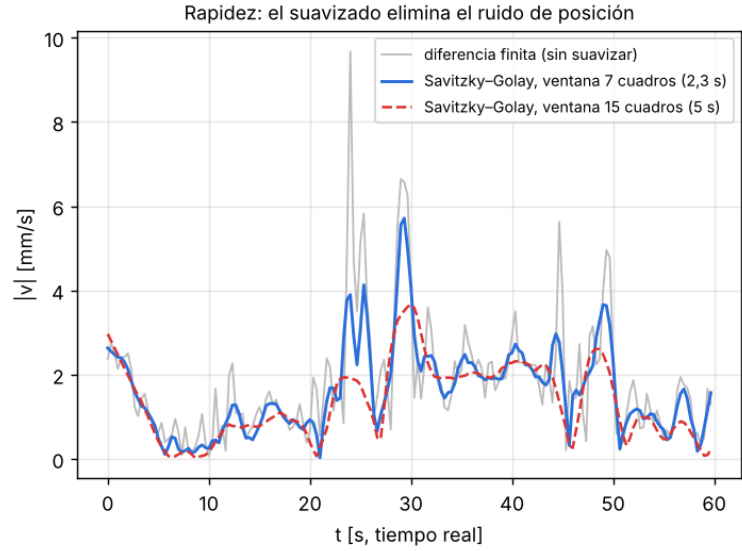
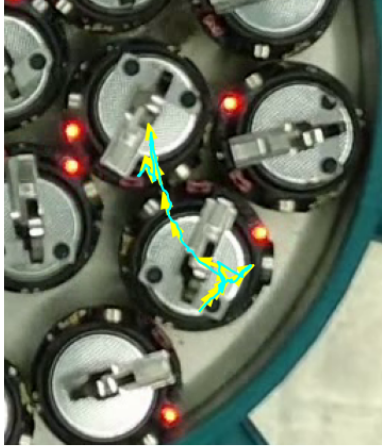


Figura 6: Trayectoria y vector velocidad de un robot (la flecha mide el desplazamiento en 7,5 cuadros, por lo que su largo no depende de f_r); rapidez en tiempo real sin suavizar y con Savitzky-Golay, con $f_r = 3$ cuadros/s.

4.7. Orientación y velocidad angular

Firma de rotación. Alrededor de cada centro la imagen se desenrolla en coordenadas polares; una rotación del robot es entonces un corrimiento a lo largo del eje angular. En dos bandas (pila, $0,15-0,70 r_{\text{ext}}$, y anillo, $0,75-0,95 r_{\text{ext}}$) se guarda el perfil angular normalizado como sus primeros $K = 12$ armónicos de Fourier (código 6). Restar la media de cada radio antes de promediar la banda elimina el gradiente radial de iluminación, y normalizar por el desvío hace a la firma insensible al brillo de la escena.

```

1  pol = cv2.warpPolar(gray, (R, 128), (x, y), R, cv2.WARP_POLAR_LINEAR) # 128 angles x R radii
2  for b, (r0, r1) in enumerate(((0.15, 0.70), (0.75, 0.95))): # battery, PCB ring
3      band = pol[:, int(r0 * R):int(r1 * R)]
4      prof = (band - band.mean(axis=0, keepdims=True)).mean(axis=1) # angular profile
5      prof = (prof - prof.mean()) / prof.std()
6      sig[b] = np.fft.fft(prof)[1:13] / 128 # harmonics 1..12

```

Código 6: Firma de rotación de un robot (core/rotation.py, resumido).

Ángulo entre dos cuadros. El giro relativo es el máximo de la correlación circular de las firmas,

$$c(\Delta) = \text{Re} \sum_b \sum_{k=1}^K S_2[b, k] \overline{S_1[b, k]} e^{ik\Delta}, \quad \Delta\theta = \arg \max_{\Delta} c(\Delta), \quad (8)$$

refinado con dos pasos de Newton (error mediano $0,37^\circ$ con rotaciones conocidas). Como $c(\Delta)$ solo tiene armónicos hasta K , una grilla de 128 puntos (por FFT inversa) ubica el máximo y la derivada analítica lo refina (código 7).

```

1 C = (s2 * np.conj(s1)).sum(axis=-2) # cross-spectrum, summed over bands
2 spec[... , 1:K + 1] = C
3 c = np.real(np.fft.ifft(spec, axis=-1)) * 128 # c(Delta) on a 128-point grid
4 d = np.argmax(c, axis=-1) * (2 * np.pi / 128)
5 k = np.arange(1, K + 1)
6 for _ in range(2): # Newton on dc/dDelta = 0
7     e = C * np.exp(1j * k * d[... , None])
8     d1 = np.real((1j * k * e).sum(-1))
9     d2 = np.real(-(k ** 2) * e).sum(-1))
10    d = d - np.where(d2 < 0, d1 / d2, 0.0)
11 angle = -((np.degrees(d) + 180) % 360 - 180) # CCW on screen (warpPolar is clockwise)

```

Código 7: Giro relativo entre dos firmas (`core/rotation.py`, resumido).

Ángulo acumulado. Sumar solo giros entre cuadros consecutivos acumula el error de cada paso. Cada cuadro se compara con los que están 1, 2, 4, 8 y 16 cuadros atrás y todos los ángulos se resuelven juntos por mínimos cuadrados ponderados por la similitud q :

$$\min_{\theta} \sum_{(i,L)} q_{i,L} (\theta_{i+L} - \theta_i - m_{i,L})^2, \quad \theta_0 = 0, \quad (9)$$

con un error acumulado $\leq 2^\circ$ en 3000 cuadros. El sistema es ralo y de banda (cada ecuación vincula dos cuadros separados a lo sumo 16), por lo que se arma con `scipy.sparse.coo_matrix` y se resuelve con las ecuaciones normales y `spsolve`, en tiempo proporcional a la cantidad de cuadros. Las comparaciones largas con similitud $q < 0,8$ o que discrepan más de 30° de la cadena incremental se descartan. Finalmente $\omega = d\theta/dt = f_r d\theta/dn$ con el mismo Savitzky–Golay (fig. 7). La fig. 8 verifica la medición: al girar cada imagen por $-\theta$, los LEDs y los puntos de la placa quedan fijos, por lo que θ describe la orientación del cuerpo del robot (la palanca de la pila gira levemente respecto de él y no sirve como referencia).

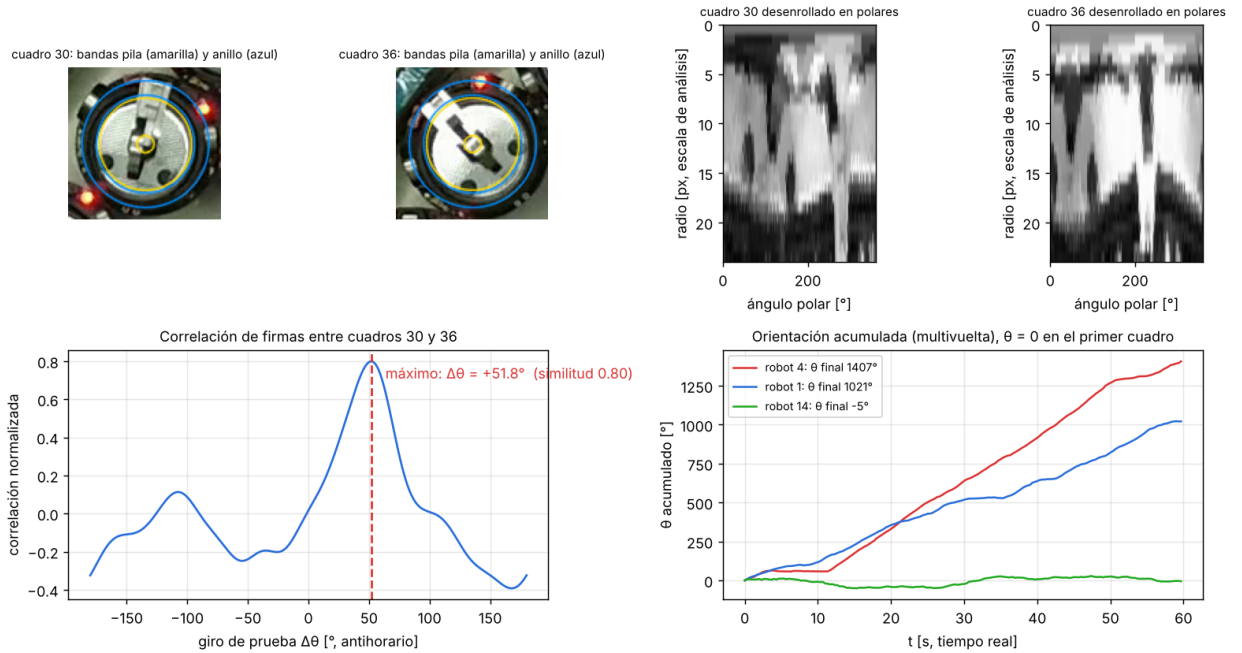


Figura 7: Firma polar en dos cuadros, correlación de las firmas y orientación acumulada de tres robots.

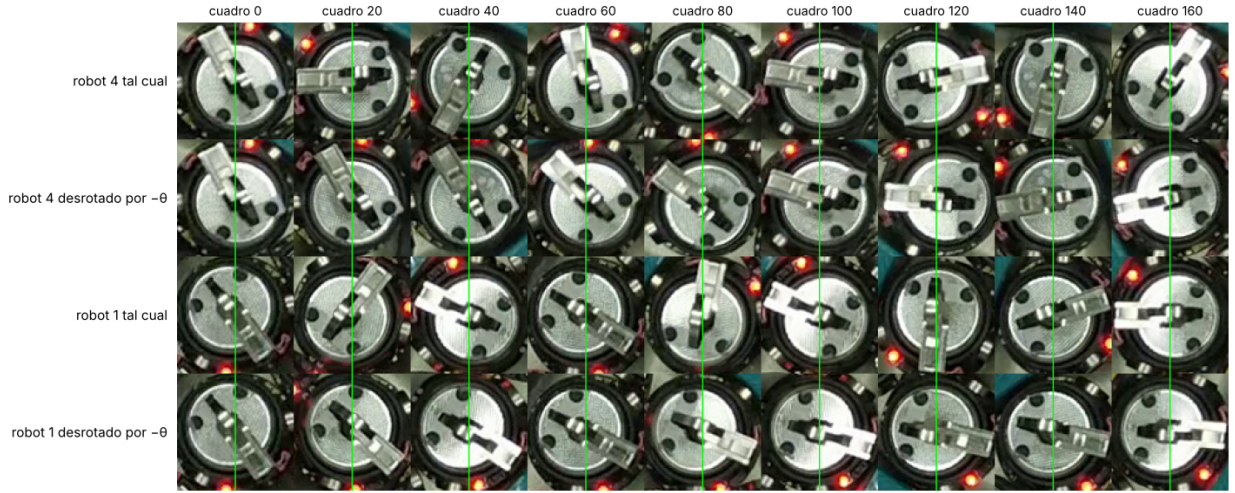


Figura 8: Validación por desrotación: filas pares, las imágenes de la fila anterior giradas por $-\theta$.

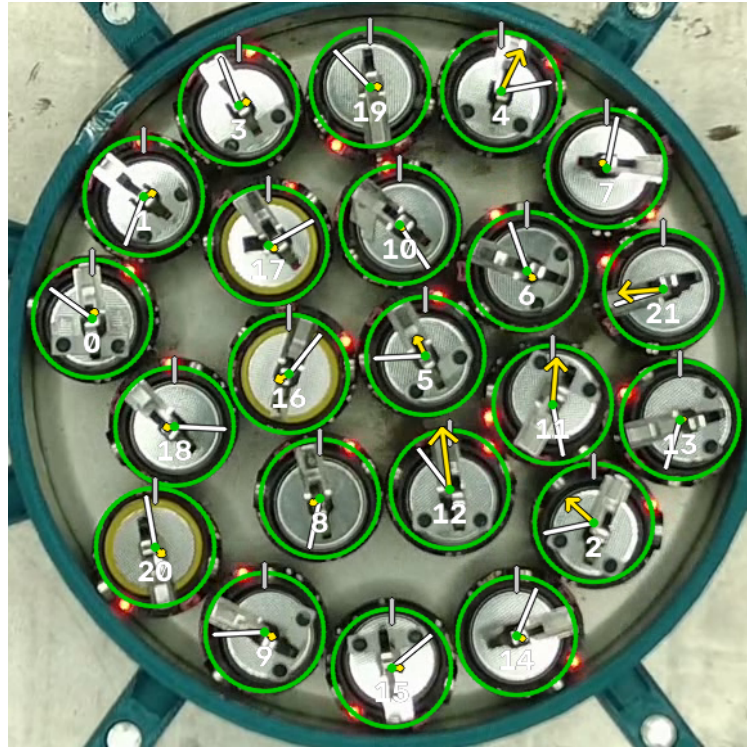


Figura 9: Superposición en la aplicación: estado (color del círculo), velocidad (flecha amarilla, desplazamiento en 7,5 cuadros) y orientación (aguja blanca; la marca a las 12 h es $\theta = 0$).

4.8. Recorte temporal

Si se recorta el tiempo después de analizar, los resultados se toman del análisis completo: las derivadas conservan su precisión en los bordes y θ se re-referencia a cero en el primer cuadro del tramo. Las exportaciones y este informe usan siempre el tramo vigente (sección 5).

4.9. Incertidumbres y limitaciones

- Posición: fluctuación de 1–5 px entre cuadros; escala espacial: $\pm 2\%$.
- Escala temporal: un error relativo en f_r pasa sin atenuar a $|\vec{v}|$ y a ω , según la ec. (3); con f_r estimado de la configuración de la cámara o del registro del sensor es del orden de 1%, con

la duración nominal del nombre puede superar el 10 %.

- Velocidad: los cambios más cortos que la ventana de suavizado (2,3 s) se atenúan, y nada más rápido que $\Delta t = 0,33$ s se resuelve.
- Orientación: $\pm 0,4^\circ$ por comparación, $\leq 2^\circ$ acumulado; en cuadros sin imagen del robot θ se interpola, y un giro mayor que media vuelta entre dos cuadros comparados no se detecta.
- Huecos: la interpolación lineal se aparta si el robot cambia de dirección; tras un hueco, un candidato aceptado con umbral relajado puede ser un falso positivo cercano a una predicción errónea. Para estadística conviene usar solo `status = detectado`.

5. Procedimiento y exportaciones

5.1. Analizar un video

1. **Abrir** el VC_<código>.MP4 (menú *Archivo*, pestaña *1 · Original*).
2. **Escala de tiempo**. En la misma pestaña, dejar marcada *Video acelerado (time-lapse)* y fijar f_r (sección 3). Si se conoce la duración real medida, usar *Calcular desde la duración real...*
3. **Recorte** (opcional, pestaña *2 · Recortar*): tramo temporal y rectángulo a analizar. Los VC ya están recortados a la arena, así que normalmente no hace falta.
4. **Seguimiento** (pestaña *4 · Seguimiento*): indicar N , verificar el diámetro real (35 mm), indicar si el video está espejado y pulsar *Analizar video*. La calibración del tamaño es automática (casilla *Calibrar tamaño automáticamente al analizar*).
5. **Revisar** los cuadros de la lista *Cuadros con problemas* (botones *Anterior* y *Siguiente*) y, si hace falta, elegir el robot y hacer clic sobre su centro real (ancla). El seguimiento se rehace en segundos, sin volver a leer el video.
6. **Guardar** el análisis (*Guardar análisis...*, archivo `.npz`) y exportar el CSV con *Exportar CSV (todos los robots)...*, formato *una fila por robot y cuadro*.

5.2. Generar el informe de ensayos

Dos scripts de `software/analisis_video/` arman el informe de resultados (`informes/ensayos/informe_ensayos.tex`) a partir de los CSV exportados. Se ejecutan desde la raíz del repositorio; en Windows la línea se continúa con `^` y en Linux o macOS con `\`. Por defecto escriben en `informes/ensayos/generado/` y leen el registro de presión de `datos/presion/crudos/`.

Una sección por ensayo. `generar_informe.py` crea `generado/<nombre>/` con la tabla de datos, la captura anotada, los gráficos y un párrafo automático, y actualiza `generado/lista_ensayos.tex`:

```
python software/analisis_video/generar_informe.py ^
    datos/video/trayectorias/VP_20262409_1600_22QR_60_robots.csv ^
    --video datos/video/recortados/VC_20262409_1600_22QR_60.MP4 ^
    --nombre 20262409_1600_22QR_60
```

Opciones:

- `--cuadros-por-segundo F`: reexpresa el CSV con $f_r = F$. Recalcula $t = n/F$ y multiplica velocidades y velocidades angulares por F/f_r^{CSV} , donde f_r^{CSV} se deduce de las columnas `frame` y `t_s`. Corrige un CSV exportado con otra escala sin repetir el análisis.

- `--imagen archivo.png --frame n`: usa una imagen ya extraída del cuadro n para la captura anotada, en lugar de leer el video (útil si el video no está en la misma máquina).
- `--titulo, --diametro-mm, --diametro-px` (si el CSV no trae escala).

Si la duración real supera 10 min, los ejes de tiempo van en minutos y las trayectorias se grafican solo para los primeros 5 min. Las observaciones propias van en `generado/<nombre>/notas.tex`, que el script nunca sobrescribe. Si el CSV se exportó con la corrección de video espejado (columna espejo), la captura se dibuja igual sobre la imagen original.

Comparación entre ensayos. `comparar_ensayos.py` recibe todos los CSV y genera `generado/comparacion/`: tabla comparativa, series temporales con los atascos marcados, distribuciones, rotación por robot, desplazamiento cuadrático medio, mapas de ocupación y, si encuentra los registros del sensor, la alineación con el registro del sensor de presión (`R_<código>.csv`) mediante el pulso LED de sincronización y la correlación cruzada:

```
python software/analisis_video/comparar_ensayos.py ^
    "datos/video/trayectorias/VP_*_robots.csv"
```

Después se compila `informes/ensayos/informe_ensayos.tex` dos veces (`pdflatex` o `latexmk -pdf`). Este informe (`informe_aplicacion.tex`) no depende de los datos.

5.3. Archivos exportados

CSV de seguimiento (una fila por robot y cuadro). Es el archivo de datos del ensayo y la entrada del informe. Tiempos, velocidades y velocidades angulares están en tiempo real, con la escala f_r vigente al exportar (tabla 7).

Tabla 7: Columnas del CSV de seguimiento.

Columna	Unidad	Contenido
<code>frame</code>	—	número de cuadro n en el archivo de video
<code>t_s</code>	s	tiempo real n/f_r
<code>particle</code>	—	identificador del robot ($0 \dots N - 1$), fijo durante todo el ensayo
<code>x_px, y_px</code>	px	centro, relativo al rectángulo analizado (y hacia abajo)
<code>status</code>	—	estado de la posición (tabla 6); <code>status_code</code> , su código numérico
<code>vx_px_s, vy_px_s</code>	px/s	componentes de la velocidad
<code>speed_px_s</code>	px/s	rapidez $ \vec{v} $
<code>vel_dir_deg</code>	°	dirección φ de $\vec{v}$, antihoraria, 0° = derecha
<code>theta_deg</code>	°	orientación acumulada θ , cero en el primer cuadro del tramo
<code>omega_deg_s, omega_rad_s</code>	°/s, rad/s	velocidad angular ω
<code>rot_status</code>	—	<i>medida, interpolada o no disponible</i>
<code>x_m, y_m</code>	m	posición en SI (si se cargó el diámetro real)
<code>vx_m_s, vy_m_s, speed_m_s</code>	m/s	velocidad en SI
<code>mass, ring_cov</code>	—	intensidad de la detección y cobertura del anillo
<code>x_video_px, y_video_px</code>	px	centro en coordenadas del video completo (sin corregir espejado)
<code>espejo</code>	—	solo si se corrigió el espejado: <code>horizontal</code> o <code>vertical</code>

CSV de seguimiento (una fila por cuadro). Mismos datos en formato ancho: una fila por cuadro y, por cada robot k , las columnas anteriores con el prefijo `rk_` (`r00_x_px, r00_y_px, ...`). Cómodo para planillas; el informe usa el formato largo.

Resumen por robot. Una fila por robot: duración (s reales), porcentaje de posiciones observadas y de rotación medida, rapidez media y máxima, recorrido total, desplazamiento neto, giro total y ω media y media en valor absoluto, en píxeles y en SI.

Video con seguimiento. El tramo analizado con la superposición de la aplicación (fig. 9): círculo coloreado por estado, número de robot, flecha de velocidad (desplazamiento en 7,5 cuadros) y aguja de orientación. Se reproduce a f_a , igual que el original.

Sesión de análisis (.npz). Guarda candidatos, firmas de rotación, anclas manuales y resultado del seguimiento, para retomar el trabajo sin volver a leer el video. Al cargarla se aplica la escala de tiempo vigente en la aplicación.